CS204: Algorithms & Data Structures Lab

Week # 04 (1 Questions, 100 Marks)
Lab session: AL1
Held on: 19-Aug-2024 (Mon)
Lab Timings: 14:00 to 17:00 Hours Pages: 3
Submission time: 16:45 Hrs, 19-Aug-2024
Instructors Dr. V. Vijaya Saradhi
Head TAs Adithya K Moorthy & Laxita Agrawal
Department of CSE, IIT Guwahati

Question 1: (100 points)

Given a matrix of a specified size, perform matrix inversion using Gaussian Elimination method. Read the supplementary material `gaussian-elimination.pdf` to understand the process of matrix inversion computation. In order to achieve this, you must perform the following tasks.

Task 01 (10 marks) Variable declaration for holding input data and reading input data:

1. Declare a class named `Matrix` with the following `private` data members
   (a) `rows` of type `int`
   (b) `columns` of type `int`
   (c) **A** of type pointer to pointer to `float`
2. Declare the following constructor with `public` access level.
   (a) (2.5 marks) Constructor with two default arguments rows and columns where the default value is 10 for rows and columns. This should initialize memory for the Matrix **A** dynamically. It should assign identity Matrix for **A**.
   (b) (2.5 marks) Constructor with three arguments namely
      i. a `const pointer to pointer to float`
      ii. an `int` representing number of rows
      iii. an `int` representing number of columns

      as input arguments. Initialize the memory for the Matrix **A** dynamically. Copy the contents of input argument into **A**.
   (c) (2.5 marks) Constructor with one argument namely a `const pointer to pointer to char` the file name as input. Initialize the memory for the Matrix **A** dynamically. Read the contents of the file into **A**.
   (d) (2.5 marks) Copy constructor which takes a `const Matrix` reference. This should initialize the memory for the Matrix **A** dynamically. Copy the contents of Matrix in this constructor.

Task 02 (10 marks) Declare a function named `augment_matrix` with the following specifications.

1. `Input: void`

2. `Output`: A Matrix object whose size is `rows` $\times 2 \times$ `columns`
3. `Functionality`: This function should augment $\mathbf{A}$ with $\mathbf{I}$ as $[\mathbf{A} \mid \mathbf{I}]$. Refer to slide number 4 of `gaussian-elimination.pdf`.

Task 03 (30 marks) Declare a function named `forward_elimination` with the following specifications.

1. `Input`: `void`
2. `Output`: `void`
3. `Functionality`: Read the forward elimination computation given in `gaussian-elimination.pd` slides. Perform these computations on augmented matrix.

Task 04 (20 marks) Declare a function named `backward_elimination` as given in `gaussian-elimination.pdf` slides

1. `Input void`
2. `Output void`
3. `Functionality` Perform the computations on the output of the Matrix obtained using `forward_elimination` method. Read the slides given in `gaussian-elimination.p` for details

Task 05 (5 marks) Declare a function named `extract_inverse` with the following specifications

1. `Input`: `void`
2. `Output`: Matrix object containing $\mathbf{A}^{-1}$
3. `Functionality`: Extract $\mathbf{A}^{-1}$ from augmented $\mathbf{A}^{-1}$ and return the matrix object after `forward_elimination` and `backward_elimination computations`

Task 06 (5 marks) Declare a `print` function with the following specifications

1. `Input`: `void`
2. `Output`: `void`
3. `Functionality`: Print the matrix and limit the precision to two decimal places.

Task 07 (5 marks) Declare a function `multiply` with the following specifications

1. `Input`: `Matrix object` $\mathbf{A}^{-1}$
2. `Output`: `Matrix object` holding the result of $\mathbf{A} \times \mathbf{A}^{-1}$
3. `Functionality`: Perform matrix multiplication of $\mathbf{A} \times \mathbf{A}^{-1}$

Task 08 (5 marks) Print the Matrix obtained from task 07.

Task 09 (5 marks) Declare a destructor with `public` access to release the dynamically allocated memory of data member within Matrix class.

Task 10 (5 marks) In the main function perform the following

1. Read the input file name from the command line argument
2. Initialize and populate Matrix object using appropriate constructor. Store this object into variable `input_matrix`
3. Perform matrix augmentation. Store it into a variable `augmented_matrix`

4. Perform `forward_elimination` operation on augmented matrix object
5. Perform `backward_elimination` operation on the augmented matrix object (only after `forward_elimination` step)
6. Perform `extract_inverse` operation on augmented matrix object. Store the resulting object into a variable `inverse`
7. Print the obtained matrix object `inverse`.
8. Perform matrix multiplication of `input_matrix` with `inverse`
9. Compute the matrix inverse on
   (a) (2 marks) `input-4x4.txt` and
   (b) (3 marks) `input-10x10.txt` respectively.